

高性能计算期末大作业（刘汇川part1的 3 4 5 6 7题目）

期末大作业实验环境

项目	详细信息
操作系统	Ubuntu 24.04.3 LTS (WSL2)
内核版本	6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core i7-11800H @ 2.30 GHz，8 物理核心 / 16 逻辑核心
内存	7.6 GiB
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB 显存) 驱动版本: 576.83 驱动支持的最高 CUDA 版本: 12.9
MPI	Open MPI 4.1.6
OpenMP	GCC 13.3.0 内置支持（OpenMP 4.5 / 5.0）
CUDA	CUDA Toolkit 12.0.140（nvcc 已安装）

第 3 题：程序正确性验证

一、题目核心要求

“基于程序中已有的算例（即平面激波向前冲击）开展计算。正式进行性能测试之前，应首先完成程序正确性验证。可以运行 CloverLeaf 提供的标准输入文件，并将最终动能与 CloverLeaf README 中给出的参考值进行比较。”

二、需要测试的四个标准算例

从 InputDecks/ 文件夹中，官方提供了 4 个推荐测试算例。从易到难：

算例文件	test_problem 编号	网格规模	时间步数	预期最终动能 (KE)	预估串行耗时
	2	960×960	87	0.1193E+01 (≈1.193)	~20 秒

clover_bm_short.in					
clover_bm.in	3	960×960	2955	0.2590E+01 (≈2.590)	~960 秒
clover_bm16_short.in	4	3840×3840	87	0.3075E+00 (≈0.3075)	~460 秒
clover_bm16.in	5	3840×3840	2955	0.4854E+01 (≈4.854)	~13000 秒

三、运行并验证

步骤 1：确认输入文件

CloverLeaf 默认读取当前目录下的 **clover.in**。直接运行：

代码块

```
1 ./clover_leaf
```

你当前的 clover.in 已经是 clover_bm_short.in 的拷贝（960×960 网格，test_problem=2），所以无需额外拷贝。

你需要关注两个点：

- 1. 动能数值：step: 87 那一行的第 7 列是 Kinetic Energy，应与 1.193E+00 非常接近
- 2. 自检结果："This test is considered PASSED" —— 出现这句话就说明通过了

```
root@DESKTOP-7GFBLHU:/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/cloverleaf/cloverleaf_serial# ./clover_leaf
Step      87 time    0.4971244 control    sound    timestep  5.85E-03      1,      1 x  5.21E-03 y  5.21E-03
Test problem 2 is within  0.1136868E-12% of the expected solution
This test is considered PASSED
Wall clock    7.9822461605072021
First step overhead  -2.8748512268066406E-003
root@DESKTOP-7GFBLHU:/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/cloverleaf/cloverleaf_serial#
```

步骤 3：源代码中的自检验证机制

field_summary.f90，其中第 154-158 行有精确的参考值：

代码块

```
1 IF(test_problem.EQ.1) qa_diff=ABS((100.0_8*(ke/1.82280367310258_8))-100.0_8)
2 IF(test_problem.EQ.2) qa_diff=ABS((100.0_8*(ke/1.19316898756307_8))-100.0_8)
3 IF(test_problem.EQ.3) qa_diff=ABS((100.0_8*(ke/2.58984003503994_8))-100.0_8)
4 IF(test_problem.EQ.4) qa_diff=ABS((100.0_8*(ke/0.307475452287895_8))-100.0_8)
```

```
5 IF(test_problem.EQ.5) qa_diff=ABS((100.0_8*(ke/4.85350315783719_8))-100.0_8)
```

程序会计算你算出的 KE 与精确参考值的百分比偏差。若偏差 < 0.001%，自动判定为 PASSED。

报告亮点：可以指出——程序中 `test_problem` 关键字触发了自检，参考值是 Intel 编译器 + IEEE 严格模式在单核上的结果。不同编译器因浮点舍入差异可能有微小偏差，但只要 `qa_diff` < 0.001% 即为通过。

步骤 4：换其他算例

代码块

```
1 cp InputDecks/clover_bm.in clover.in          # 换 test_problem 3
2 ./clover_leaf                                # 预期 KE ≈ 2.590
3
4 cp InputDecks/clover_bm16_short.in clover.in  # 换 test_problem 4
5 ./clover_leaf                                # 预期 KE ≈ 0.3075
6
7 cp InputDecks/clover_bm16.in clover.in        # 换 test_problem 5
8 ./clover_leaf                                # 预期 KE ≈ 4.854
```

```
Wall clock    309.78517198562622
Average time per cell    1.1379093583736293E-007
Step time per cell    1.2791017070412636E-007
Step   2955 time  15.4982864 control    sound    timestep    5.27E-03          1,          1 x  5.21E-03 y  5.21E-03
Test problem   3 is within  0.1787088E-01% of the expected solution
This test is considered NOT PASSED
Wall clock    309.90967988967896
First step overhead  -2.9320716857910156E-003
root@DESKTOP-7GFB LHU:/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/cloverleaf/cloverleaf_serial# |
```

新电脑的数据

```
Time  15.503561035124122
      Volume    Mass    Density    Pressure Internal Energy Kinetic Energy  Total Energy
step: 2955  0.1000E+03  0.2800E+02  0.2800E+00  0.1604E+00  0.4009E+02  0.2589E+01  0.4268E+02
```

Test problem 3 is within 0.1787088E-01% of the expected solution
This test is considered NOT PASSED

Calculation complete
Clover is finishing
Wall clock 342.76962804794312
First step overhead 8.0895423889160156E-004

```
Wall clock    128.09824705123901
Average time per cell    1.0101416697833947E-007
Step time per cell    9.4968951695288215E-008
Step    87 time   0.1242811 control    sound    timestep    1.46E-03          1,          1 x  1.30E-03 y  1.30E-03
Test problem   4 is within  0.1580247E-10% of the expected solution
This test is considered PASSED
Wall clock    129.57828783988953
First step overhead  -9.4849824905395508E-002
root@DESKTOP-7GFB LHU:/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/cloverleaf/cloverleaf_serial# |
```

```
Wall clock 67.205014944076538
Average time per cell 5.2995718189861710E-008
Step time per cell 5.1526892800918886E-008
Step 87 time 0.1242811 control sound timestep 1.46E-03 1, 1 x 1.30E-03 y 1.30E-03
Test problem 4 is within 0.3326761E-10% of the expected solution
This test is considered PASSED
Wall clock 68.009786844253540
First step overhead -4.2913913726806641E-002
liuhuichuan@DESKTOP-650U451:/mnt/c/Users/admin/Desktop/CloverLeaf/CloverLeaf_ref$

Wall clock 8655.3859980106354
Average time per cell 1.9870715293486847E-007
Step time per cell 1.9495381921943690E-007
Step 2955 time 4.3502757 control sound timestep 1.49E-03 1, 1 x 1.30E-03 y 1.30E-03
Test problem 5 is within 0.4884413E-09% of the expected solution
This test is considered PASSED
Wall clock 8658.4761531352997
First step overhead -3.1929969787597656E-002
root@DESKTOP-7GFB LHU:/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/cloverleaf/cloverleaf_serial# |
```

四、发现

根本原因

`-ffloat-store` 的设计目的是解决旧 x86 架构（32 位）上 x87 FPU 的 80 位中间精度问题。但你的 WSL 运行在 **x86-64** 上，gfortran 默认使用 **SSE/SSE2 指令集** 做浮点运算（本身就是 64 位），所以这个标志基本不起作用。

gfortran 和 Intel ifort 在**浮点运算顺序**、**FMA（乘加融合）**、**数学库函数**（如 `fabs` / `sqrt`）实现上的差异，是任何 IEEE 标志都无法消除的。这是官方明确说明的：

"Even with the IEEE options set, this is not guarantee that different compilers or platforms will produce the same answers."

test problem 3 自检分析：

在 GNU 编译器（gfortran 13.x + gcc 13.x）下，test problem 3 的动能偏差为 0.018%，略超程序设定的 0.001% 自检阈值。经分析，该偏差源于以下三个层面的编译器差异：

- 1. **浮点运算顺序**：gfortran 与 Intel ifort 在代数化简和运算重排策略上不同，导致舍入误差累积路径不同；
- 2. **FMA 指令使用**：Intel 编译器默认融合乘加（`a*b+c` 用一条指令完成，只舍入一次），gfortran 可能分成两条指令（舍入两次）；
- 3. **数学库实现**：`fabs`、`sqrt` 等超越函数的实现精度在不同编译器间存在微小差异。

由于 test 3 步数多达 2955 步，微小差异随迭代逐步放大。实测最终动能约 2.590，与参考值 2.58984 的相对偏差仅 0.018%，在实际 CFD 计算中完全可忽略。该现象印证了 CloverLeaf README 第 103-109 行的说明——不同编译器不保证 bitwise identical 结果。test 2（87 步）、test 4（87 步）、test 5（2955 步）均以 $10^{-10} \sim 10^{-13}$ 量级的极小偏差通过，进一步证明程序在物理上计算正确。

最终结果汇总

算例	网格	步数	偏差	结果

test 2	960×960	87	$1.14 \times 10^{-13}\%$	✓ PASSED
test 3	960×960	2955	$1.79 \times 10^{-2}\%$	NOT PASSED
test 4	3840×3840	87	$1.58 \times 10^{-11}\%$	✓ PASSED
test 5	3840×3840	2955	$4.88 \times 10^{-10}\%$	✓ PASSED

第 4 题：不同网络规模 and 不同版本的验证

4.1 最基本的串行运行的时间

```
cd "/mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/CloverLeaf/CloverLeaf_Serial"
cp InputDecks/clover_bm4_short.in clover.in
echo "=== Serial ===" && ./clover_leaf
=== Serial ===

Clover Version      1.300
      MPI Version
      Task Count    1

Average time per cell  1.2697984157033862E-007
Step time per cell    1.2202123697433206E-007
Step      87 time      0.2485622 control      sound      timestep  2.92E-03      1,      1 x  2.60E-03 y  2.60E-03
Wall clock    40.745328903198242
First step overhead -9.1681480407714844E-003
root@DESKTOP-7GFB LHU: /mnt/f/a研究生/博一下的课程/高性能计算/作业/期末作业/CloverLeaf/CloverLeaf_Serial#
```

4.2 CloverLeaf 并行性能测试报告（小规模用例 clover_bm4_short.in）

1. 全部测试结果汇总

版本	并行配置	总路数	总运行时间 (s)	加速比	并行效率	备注
Serial	1 核	1	40.745	1.00	100%	基准
OpenMP	2 线程	2	35.508	1.15	57.4%	
OpenMP	4 线程	4	36.104	1.13	28.2%	
OpenMP	8 线程	8	38.301	1.06	13.3%	
MPI	2 进程	2	26.420	1.54	77.1%	最快
MPI	4 进程	4	28.904	1.41	35.2%	
MPI	8 进程	8	32.475	1.25	15.7%	
MPI+OpenMP	1 进程 × 4 线程	4	29.008	1.40	35.1%	

MPI+OpenMP	2 进程 × 2 线程	4	30.966	1.32	32.9%	
MPI+OpenMP	2 进程 × 4 线程	8	28.407 ⚠	1.43 ⚠	17.9% ⚠	计时异常

注：加速比 = 串行时间 / 并行时间；并行效率 = 加速比 / 总并行路数。

⚠ 2 进程 × 4 线程 组在运行中途 Wall clock 出现负值，计时失真，相关指标仅供参考。

2 关键发现

整体加速效果有限

- 在本测试用例上，最高加速比仅为 **1.54**（MPI-2 进程），远未达到线性加速。
- 所有配置的并行效率均低于 80%，多数在 30% 左右甚至更低。

加速比随并行度增加不升反降

- OpenMP：2 线程最快，4 线程、8 线程反而变慢。
- MPI：2 进程最快，4 进程、8 进程逐步降速。
- 混合模式：1×4 和 2×2 均不如纯 MPI-2。
- 说明**并行引入的额外开销（线程管理、同步、通信）已超过并行带来的计算收益**。

纯 MPI 小规模下优于纯 OpenMP

- MPI-2 进程（26.4 s）显著优于 OpenMP-2 线程（35.5 s）。
- 可能原因：MPI 独立地址空间避免伪共享，通信开销在小进程数下可控。

计时不稳定问题

- 多处出现 Wall clock 不增反降、Step time per cell 为负、First step overhead 为负。
- 在 MPI+OpenMP 混合、较高并行度时尤为严重。
- 原因：WSL 环境下计时的非单调性（`gettimeofday` / `clock_gettime` 受虚拟化影响）。

3 原因分析

问题规模过小（最主要原因）

- `clover_bm4_short.in` 为微型测试，单步每网格计算量极低。
- 每个并行单元分得的工作量小于并行开销本身，强扩展性注定极差。

内存带宽瓶颈

- CloverLeaf 为显式流体力学求解器，属内存受限型应用。

- 多核/多进程并发访问主存时，共享缓存与内存带宽竞争加剧，单核有效带宽下降。

负载不均衡与伪共享

- OpenMP 并行区域可能存在迭代分配不均，隐式 barrier 增加同步等待。
- 多线程共享数组时，相邻线程写入同一 cache line 导致伪共享，降低有效带宽。

运行环境因素

- WSL 的 CPU 调度与计时精度不如原生 Linux，可能放大性能波动。
- 计时器不单调导致个别步出现负值，影响局部分析但不改变总体趋势。

4.3 Ref 版（MPI+OpenMP）长算例并行效率测试

选定长步数系列（2955 步）的三个算例（bm2、bm4、bm8），在 Ref 版上以基准配置 np=1、OMP=1 获取单核串行基准 T_1 ，并以 np=2、OMP=4（共 8 核）获取并行时间 T_8 。

1 Ref 版单核基准与 8 核并行效率对比（2955 步）

算例	网格	单元数	单核 T_1 (s)	8 核 T_8 (s)	加速比 S_8	效率 E_8
bm2	1920×960	1.84×10^6	654.7	528.1	1.24	15.5%
bm4	1920×1920	3.69×10^6	1247.8	1146.9	1.09	13.6%
bm8	3840×1920	7.37×10^6	2498.6	2108.2	1.19	14.9%

三个规模下 8 核并行效率均仅约 14%–16%，远低于理想值 100%。更为关键的是，即使网格从 1.84M 扩大到 7.37M（4 倍），效率并未显著改善，始终被压制在 ~15% 附近。这一现象具有深刻的体系结构根源——CloverLeaf 是典型的 **Memory Bandwidth Bound（内存带宽受限）** 型应用。

Memory Bound 成因分析

`advec_cell_kernel.c.c` 中存在大量如下形式的数组访问模式：

代码块

```
1 pre_vol[...] = volume[...] + vol_flux_x[...] - vol_flux_x[...]
2               + vol_flux_y[...] - vol_flux_y[]...];
3 mass_flux_x[...] = vol_flux_x[...] * (density1[...] + limiter);
4 post_ener[...] = (energy1[...] * pre_mass[...] + ener_flux[...]
5                  - ener_flux[...]) / post_mass[...];
```

在每个网格单元上，上述代码的算术操作（加减乘除）仅十余条，而访存操作（读写 `volume`、`vol_flux_x/y`、`density1`、`energy1`、`pre_mass`、`post_mass`、`ener_flux` 等数

组) 高达数十次。该程序的计算访存比 (Operational Intensity) 极低——通常不足 0.1 FLOP/byte。当代 CPU 的单核内存带宽约为其峰值浮点性能的 1/15~1/20。在 CloverLeaf 中, 单核已接近内存带宽利用上限。启动额外核心 (无论是 MPI 多进程还是 OpenMP 多线程) 时, 所有计算单元共享同一组内存通道 (本例为双通道 DDR4, 理论带宽约 51.2 GB/s), 而实际有效带宽远低于此值。多核并发请求形成排队争用, 导致每个核心等待数据的时延大幅增加, 计算单元大量空转。并行效率因此被"内存墙 (Memory Wall)"锁死在低位。

弱可扩展性分析

尽管绝对效率偏低, 每单元每步计算时间在不同网格规模下保持高度稳定:

- bm2: 9.70×10^{-8} 秒/单元/步
- bm4: 1.05×10^{-7} 秒/单元/步 (略高约 8%)
- bm8: 9.68×10^{-8} 秒/单元/步

bm4 的轻微偏离可能是因为它是唯一方形网格 (1920×1920), 其 MPI 子域的周长/面积比大于长方形网格 (bm2 的 1920×960、bm8 的 3840×1920), 导致 halo 交换数据量相对更大。总体而言, 本测试范围内弱可扩展性良好——问题规模扩大 4 倍, 每单元计算成本基本不变。

4.5 同规模 (bm8) 下 MPI、OpenMP 与 Ref 混合版本的横向对比

为揭示不同并行化模型在 Memory Bound 条件下的效率差异, 选取最大规模算例 bm8 (3840×1920, 2955 步) 进行跨版本对比。

bm8 长算例各版本耗时与并行效率

版本	配置	核心数 p	耗时 (s)	S_p	E_p
Ref (基准)	np=1, OMP=1	1	2498.6	1.00	100%
MPI	np=4	4	2180.4	1.15	28.7%
MPI	np=8	8	2191.5	1.14	14.3%
OpenMP	OMP=4	4	2315.2	1.08	27.0%
OpenMP	OMP=8	8	2968.8	0.84	10.5%
Ref 混合	np=2, OMP=4	8	2108.2	1.19	14.9%

结论一：OpenMP 8 线程出现"负加速"——内存带宽饱和的直接证据

OpenMP 8 线程耗时 (2968.8 s) 超过单核基准 (2498.6 s), 加速比仅为 0.84, 即并行后反而慢了 16%。这是 Memory Bandwidth Saturation 的经典表现: 在线程数超越物理核心所对应的内存通道供给能力后, 每个线程不仅无法获得额外带宽, 还因以下因素产生额外开销:

- **伪共享 (False Sharing)**：多个线程写入同一 cache line 内的不同变量，触发无意义的 cache 一致性协议 (MESI) 失效；
- **线程调度开销**：OpenMP 的 `!$OMP PARALLEL DO` 在每 kernel 入口处的 `fork` 与出口处的 `join` 引入同步开销，在 Memory Bound 场景下无法被计算时间的缩短所抵消；
- **内存控制器排队拥堵**：8 个线程同时向内存控制器发出读写请求，排队深度增加导致有效时延上升。

结论二：MPI 系统地优于 OpenMP——进程隔离与 NUMA 亲和性的优势

在同等 4 核下，MPI (2180.4 s) 比 OpenMP (2315.2 s) 快约 6%；在同等 8 核下，MPI (2191.5 s) 比 OpenMP (2968.8 s) 快约 35%。这种差异随着核心数增大而急剧扩大。

根本原因在于：

1. **独立地址空间**：MPI 各进程拥有独立的虚拟地址空间，操作系统的物理页分配器可以将不同进程的数据分布到不同内存通道/NUMA 节点，实现跨通道并行；OpenMP 线程共享同一地址空间，所有数据集中分配，无法享受多通道的并行带宽。
2. **无伪共享**：MPI 进程间无共享内存，从根本上杜绝了伪共享问题。
3. **通信显式可控**：MPI 的 halo 交换（通过 `update_halo.f90`）使用 `MPI_Sendrecv` 的点对点通信，由 MPI 库在底层优化；OpenMP 的数据共享隐式依赖硬件 cache 一致性协议，在边界数据更新时可能触发大量不必要的缓存失效。

结论三：Ref 混合并行 (2 MPI × 4 OMP) 为最优配置

在全部 8 核组合中，Ref (np=2, OMP=4) 以 2108.2 s 取得最快成绩，同时是在 8 核下唯一一种跨版本的最优实践。其背后的原理是：

- **进程数适度** (np=2)：避免 8 个独立进程激烈争抢内存带宽；
- **线程数适度** (OMP=4)：每个进程内 4 个线程（恰好等于单进程所能调度的理想带宽上限），在保持一定计算密度的同时未触发线程间的伪共享与调度风暴；
- **二者互补**：MPI 负责跨内存通道的粗粒度带宽分配，OpenMP 负责单通道内的细粒度计算拆分，形成了层次化的“通道感知”并行策略。

4.6 总结

CloverLeaf 作为典型的显式流体力学 mini-app，其并行效率的根本瓶颈在于内存带宽而非计算能力。在单节点 8 核环境下，即使采用最优的混合并行配置，加速比也仅约 1.2，效率约 15%。该结果与 README 的性能参考表（单节点 16 核约可取得 5 倍以上的加速比）存在显著差异——一台高带宽服务器（如支持 8 通道内存的 Intel Xeon 平台）中的多通道内存架构能够为更多核心提供足够的聚合带宽，而笔记本或桌面级的双通道 DDR4 平台则无法支撑本实验中的多核并行需求。这一发现有力地证明了：对于 Memory Bound 型高性能计算应用，内存带宽和通道数是决定并行效率的首要硬件因素，其优先级甚至高于核心数量。在类似场景中进行并行优化时，**应先考虑提升内存通道数、优化数据布局和使用 NUMA 亲和性绑定，再考虑增加核心数。**

项目	当前配置信息
操作系统	Ubuntu 24.04.3 LTS (WSL2)
内核版本	6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core Ultra 7 265K @ 3.90GHz (20 物理核心 / 20 逻辑核心)
内存	15 GiB
GPU	未安装 NVIDIA 驱动 (无法使用 CUDA/GPU)
MPI	Open MPI 4.1.6
OpenMP	GCC 13.3.0 内置支持 (OpenMP 4.5 / 5.0)
CUDA	未安装 (用户声明无需 CUDA)

serial

```
=== Serial bm_short ===
Step      87 time    0.4971244 control    sound    timestep  5.85E-03      1,      1 x  5.21E-03 y  5.21E-03
Test problem  2 is within  0.1136868E-12% of the expected solution
This test is considered PASSED
Wall clock    5.3336288928985596
First step overhead -3.6993026733398438E-002
=== Serial bm4_short ===
Average time per cell    6.8295050053845236E-008
Step time per cell    7.1860073755184807E-008
Step      87 time    0.2485622 control    sound    timestep  2.92E-03      1,      1 x  2.60E-03 y  2.60E-03
Wall clock    21.914438962936401
First step overhead  7.4653625488281250E-003
```

openmp

```

=== OpenMP OMP=1 ===
Average time per cell    6.6841425776038830E-008
Step time per cell      6.6579862808187798E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    21.447977066040039
First step overhead    2.2176980972290039E-002
---
=== OpenMP OMP=2 ===
Average time per cell    5.3128431844753660E-008
Step time per cell      5.2737046240104571E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    17.050788879394531
First step overhead    -1.5940666198730469E-003
---
=== OpenMP OMP=4 ===
Average time per cell    4.8462620807585405E-008
Step time per cell      4.8149959184229376E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.553333997726440
First step overhead    -3.7097930908203125E-003
---
=== OpenMP OMP=8 ===
Average time per cell    4.8428913758879944E-008
Step time per cell      4.8169491088224782E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.541527032852173
First step overhead    -3.7109851837158203E-003
---
=== OpenMP OMP=10 ===
Average time per cell    4.8798730943438617E-008
Step time per cell      4.8561487346887587E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.658610105514526
First step overhead    -6.6440105438232422E-003
---
=== OpenMP OMP=16 ===
Average time per cell    4.9386708034582685E-008
Step time per cell      4.9033422126538221E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.849040031433105
First step overhead    -3.3020973205566406E-004
---
=== OpenMP OMP=20 ===
Average time per cell    5.0269203857357498E-008
Step time per cell      4.9800857798092893E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    16.133152961730957
First step overhead    -3.9601325988769531E-004
---

```

mpi

```

=== MPI np=1 ===
Average time per cell    6.5077714849436698E-008
Step time per cell      6.4573832787573331E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    20.896282911300659
First step overhead  -3.3099651336669922E-003
---
=== MPI np=2 ===
Average time per cell    5.3144748790733496E-008
Step time per cell      5.2049937140610483E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    17.053442001342773
First step overhead    3.5340785980224609E-003
---
=== MPI np=4 ===
Average time per cell    4.7504993484088384E-008
Step time per cell      4.7180478254126179E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.241699934005737
First step overhead    9.3727111816406250E-003
---
=== MPI np=8 ===
Average time per cell    4.7736968309678695E-008
Step time per cell      4.6096198881665863E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.320242166519165
First step overhead    5.7730674743652344E-003
---
=== MPI np=10 ===
Average time per cell    4.7818704951044356E-008
Step time per cell      4.7804917105370098E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.347197055816650
First step overhead    6.7172050476074219E-003
---
=== MPI np=16 ===
Average time per cell    4.8830266109395781E-008
Step time per cell      5.0209928303956987E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.676479816436768
First step overhead    8.9471340179443359E-003
---
=== MPI np=20 ===
Average time per cell    4.9346810217600227E-008
Step time per cell      5.0334169322417841E-008
Step    87 time    0.2485622 control    sound    timestep    2.92E-03    1,    1 x  2.60E-03 y  2.60E-03
Wall clock    15.849137067794800
First step overhead    9.3131065368652344E-003
---

```

openmp+mpi

```

=== Ref np=1 OMP=4 (p=4) ===
Average time per cell 6.6979100430764507E-008
Step time per cell 6.7700489631129640E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 21.494683027267456
First step overhead -1.3210773468017578E-003
---
=== Ref np=2 OMP=2 (p=4) ===
Average time per cell 5.4772447816334491E-008
Step time per cell 5.4282783013251093E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 17.576531887054443
First step overhead 8.6770057678222656E-003
---
=== Ref np=4 OMP=1 (p=4) ===
Average time per cell 4.9698404042478605E-008
Step time per cell 4.9578957259654998E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 15.947413921356201
First step overhead -2.1469593048095703E-003
---
=== Ref np=2 OMP=4 (p=8) ===
Average time per cell 5.5217679267513352E-008
Step time per cell 5.4789189663198262E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 17.716743946075439
First step overhead -3.1788349151611328E-003
---
=== Ref np=4 OMP=2 (p=8) ===
Average time per cell 4.9990332006772822E-008
Step time per cell 5.0475807963973948E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 16.048449039459229
First step overhead -5.6121349334716797E-003
---
=== Ref np=2 OMP=5 (p=10) ===
Average time per cell 5.5730529370156988E-008
Step time per cell 5.6508773316939675E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 17.891709089279175
First step overhead -5.1474571228027344E-004
---
=== Ref np=5 OMP=2 (p=10) ===
Average time per cell 5.0200195441585637E-008
Step time per cell 5.1931452213062180E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 16.118070840835571
First step overhead -2.3751258850097656E-003
---

```

```

=== Ref np=4 OMP=4 (p=16) ===
Average time per cell 5.1005994774718851E-008
Step time per cell 4.8239210930963358E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 16.351857900619507
First step overhead 7.8201293945312500E-004
---
=== Ref np=2 OMP=8 (p=16) ===
Average time per cell 5.7078576682842977E-008
Step time per cell 6.2500799281729588E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 18.317362070083618
First step overhead -1.1116027832031250E-002
---
=== Ref np=4 OMP=5 (p=20) ===
Average time per cell 5.0969932346246195E-008
Step time per cell 5.0115825918813547E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 16.354449987411499
First step overhead -2.3920536041259766E-003
---
=== Ref np=5 OMP=4 (p=20) ===
Average time per cell 5.0975865917881945E-008
Step time per cell 5.2915614408751331E-008
Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03
Wall clock 16.361485958099365
First step overhead -1.9216537475585938E-004

```

不同网格规模测试完成

echo "===== 不同网格规模测试完成 ====="

echo "结果文件："

ls -la "\$RES/grids/"

=== Ref clover_bm2_short np=4 OMP=1 (最优4核) ===

Average time per cell 4.7315802793343401E-008

Step time per cell 4.5772046885556644E-008

Step 87 time 0.2488774 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 5.21E-03

Wall clock 7.5924799442291260

First step overhead 1.2149810791015625E-003

=== Ref clover_bm2_short np=4 OMP=5 (全20核) ===

Average time per cell 4.8388048070326484E-008

Step time per cell 4.8082761673463715E-008

Step 87 time 0.2488774 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 5.21E-03

Wall clock 7.7651059627532959

First step overhead 1.9407272338867188E-004

=== Ref clover_bm4_short np=4 OMP=1 (最优4核) ===

Average time per cell 4.8984475800820484E-008

Step time per cell 5.0797761004004212E-008

Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03

Wall clock 15.727448940277100

First step overhead 5.9604644775390625E-004

=== Ref clover_bm4_short np=4 OMP=5 (全20核) ===

Average time per cell 4.9465738846956526E-008

Step time per cell 4.9186184898846681E-008

Step 87 time 0.2485622 control sound timestep 2.92E-03 1, 1 x 2.60E-03 y 2.60E-03

Wall clock 15.871319055557251

First step overhead -1.6841888427734375E-003

=== Ref clover_bm8_short np=4 OMP=1 (最优4核) ===

Average time per cell 4.9305894894607232E-008

Step time per cell 4.9271265096548528E-008

Step 87 time 0.1244387 control sound timestep 1.46E-03 1, 1 x 1.30E-03 y 2.60E-03

Wall clock 31.644527196884155

First step overhead 3.9899349212646484E-003

=== Ref clover_bm8_short np=4 OMP=5 (全20核) ===

Average time per cell 4.9856305483353231E-008

Step time per cell 4.9280578322294687E-008

Step 87 time 0.1244387 control sound timestep 1.46E-03 1, 1 x 1.30E-03 y 2.60E-03

Wall clock 31.997352123260498

First step overhead -3.3898353576660156E-003

=== Ref clover_bm16_short np=4 OMP=1 (最优4核) ===

Step 87 time 0.1242811 control sound timestep 1.46E-03 1, 1 x 1.30E-03 y 1.30E-03

Wall clock 63.801892042160034

First step overhead -1.1360883712768555E-002

Test problem 4 is within 0.3301182E-10% of the expected solution

This test is considered PASSED

=== Ref clover_bm16_short np=4 OMP=5 (全20核) ===

Step 87 time 0.1242811 control sound timestep 1.46E-03 1, 1 x 1.30E-03 y 1.30E-03

Test problem 4 is within 0.5042011E-10% of the expected solution

This test is considered PASSED

Wall clock 64.386543989181519

First step overhead -7.6899528503417969E-003

=== Ref clover_bm2 np=4 OMP=1 (最优4核) ===

Average time per cell 4.7401335275850592E-008

Step time per cell 4.7218959985507857E-008

Step 2955 time 8.9432113 control sound timestep 2.64E-03 1, 1 x 2.60E-03 y 5.21E-03

Wall clock 258.18300890922546

First step overhead -3.2572746276855469E-003

=== Ref clover_bm2 np=4 OMP=5 (全20核) ===

Average time per cell 4.8640441027356645E-008

Step time per cell 4.8529667158921561E-008

Step 2955 time 8.9432113 control sound timestep 2.64E-03 1, 1 x 2.60E-03 y 5.21E-03

Wall clock 264.93248987197876

First step overhead 3.0119419097900391E-003

=== Ref clover_bm4 np=4 OMP=1 (最优4核) ===

Average time per cell 4.9005505109695849E-008

Step time per cell 4.8541114665567872E-008

Step 2955 time 8.4089430 control sound timestep 2.74E-03 1, 1 x 2.60E-03 y 2.60E-03

Wall clock 533.84089589118958

First step overhead 8.1579685211181641E-003

=== Ref clover_bm4 np=4 OMP=5 (全20核) ===

Average time per cell 4.9980529079200150E-008

Step time per cell 4.9440617052217324E-008

Step 2955 time 8.4089430 control sound timestep 2.74E-03 1, 1 x 2.60E-03 y 2.60E-03

Wall clock 544.46371006965637

First step overhead 4.0180683135986328E-003

=== Ref clover_bm8 np=4 OMP=1 (最优4核) ===

Average time per cell 4.9621166593352368E-008

Step time per cell 5.0108776324325141E-008

Step 2955 time 4.4464018 control sound timestep 1.59E-03 1, 1 x 1.30E-03 y 2.60E-03

Wall clock 1081.0951459407806

First step overhead 5.9080123901367188E-004

=== Ref clover_bm8 np=4 OMP=5 (全20核) ===

Average time per cell 5.0001776090556254E-008

Step time per cell 4.9561753662096129E-008

Step 2955 time 4.4464018 control sound timestep 1.59E-03 1, 1 x 1.30E-03 y 2.60E-03

Wall clock 1089.3926661014557

First step overhead 1.1493206024169922E-002

===== 不同网格规模测试完成 =====

结果文件：

total 16

drwxrwxrwx 1 liuhuichuan liuhuichuan 4096 Jun 9 20:22 .

drwxrwxrwx 1 liuhuichuan liuhuichuan 4096 Jun 9 20:14 ..

-rwxrwxrwx 1 liuhuichuan liuhuichuan 1041 Jun 9 2026 long_20core.txt

-rwxrwxrwx 1 liuhuichuan liuhuichuan 1047 Jun 9 21:03 long_4core.txt

-rwxrwxrwx 1 liuhuichuan liuhuichuan 1414 Jun 9 20:18 short_20core.txt

-rwxrwxrwx 1 liuhuichuan liuhuichuan 1422 Jun 9 20:17 short_4core.txt

liuhuichuan@DESKTOP-650U451:/mnt/c/Users/admin/Desktop/CloverLeaf/CloverLeaf_ref\$

当然可以。以下是根据您提供的测试数据整理的 **Markdown 文档**，可直接保存为 `.md` 文件。

更换硬件设备测试

1. 系统配置

项目	信息
操作系统	Ubuntu 24.04.3 LTS (WSL2)
内核版本	6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core Ultra 7 265K @ 3.90GHz (20 物理核心 / 20 逻辑核心)
内存	15 GiB
GPU	未安装 NVIDIA 驱动 (无法使用 CUDA/GPU)
MPI	Open MPI 4.1.6
OpenMP	GCC 13.3.0 内置支持 (OpenMP 4.5 / 5.0)
CUDA	未安装 (用户声明无需 CUDA)

2. 纯 MPI 性能测试

MPI np	Average time per cell (s)	Step time per cell (s)	Step 87 time (s)	Wall clock (s)	First step overhead (s)
1	5.3145E-8	5.2050E-8	0.24856	17.05344	3.5341E-3
4	4.7505E-8	4.7180E-8	0.24856	15.24170	9.3727E-3
8	4.7737E-8	4.6096E-8	0.24856	15.32024	5.7731E-3
10	4.7819E-8	4.7885E-8	0.24856	15.34720	6.7172E-3
16	4.8830E-8	5.0210E-8	0.24856	15.67648	8.9471E-3
20	4.9347E-8	5.0334E-8	0.24856	15.84914	9.3131E-3

3. 混合 MPI+OpenMP 性能测试

配置 (np × OMP)	总核数	Average time per cell (s)	Step time per cell (s)	Step 87 time (s)	Wall clock (s)	First step overhead (s)
np=1 OMP=4	4	6.6979E-8	6.7700E-8	0.24856	21.49468	-1.3211E-3
np=2 OMP=2	4	5.4772E-8	5.4283E-8	0.24856	17.57653	8.6770E-3
np=4 OMP=1	4	4.9698E-8	4.9579E-8	0.24856	15.94741	-2.1470E-3

np=2 OMP=4	8	5.5218E-8	5.4789E-8	0.24856	17.71674	-3.1788E-3
np=4 OMP=2	8	4.9993E-8	5.0476E-8	0.24856	16.04845	-5.6121E-3
np=2 OMP=5	10	5.5731E-8	5.6509E-8	0.24856	17.89171	-5.1475E-4
np=5 OMP=2	10	5.0200E-8	5.1931E-8	0.24856	16.11807	-2.3751E-3
np=4 OMP=4	16	5.1006E-8	4.8239E-8	0.24856	16.35186	7.8201E-4
np=2 OMP=8	16	5.7979E-8	6.2501E-8	0.24856	18.31736	-1.1116E-2
np=4 OMP=5	20	5.0970E-8	5.0116E-8	0.24856	16.35445	-2.3921E-3
np=5 OMP=4	20	5.0976E-8	5.2916E-8	0.24856	16.36149	-1.9217E-4

4. 不同网格规模测试

所有测试对比了 **np=4 OMP=1（最优4核）** 与 **np=4 OMP=5（全20核）** 两种配置。

4.1 bm2_short（网格尺寸：1×2.60E-3 × 5.21E-3）

配置	Average time per cell (s)	Step time per cell (s)	Step 87 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.7316E-8	4.5772E-8	0.24888	7.59248	1.2150E-3
np=4 OMP=5	4.8388E-8	4.8083E-8	0.24888	7.76511	1.9407E-4

4.2 bm4_short（网格尺寸：1×2.60E-3 × 2.60E-3）

配置	Average time per cell (s)	Step time per cell (s)	Step 87 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.8984E-8	5.0798E-8	0.24856	15.72745	5.9605E-4
np=4 OMP=5	4.9466E-8	4.9186E-8	0.24856	15.87132	-1.6842E-3

4.3 bm8_short（网格尺寸：1×1.30E-3 × 2.60E-3）

配置	Average time per cell (s)	Step time per cell (s)	Step 87 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.9306E-8	4.9271E-8	0.12444	31.64453	3.9899E-3
np=4 OMP=5	4.9856E-8	4.9281E-8	0.12444	31.99735	-3.3898E-3

4.4 bm16_short (网格尺寸: 1×1.30E-3 × 1.30E-3)

配置	Step 87 time (s)	Wall clock (s)	First step overhead (s)	测试状态
np=4 OMP=1	0.12428	63.80189	-1.1361E-2	PASSED
np=4 OMP=5	0.12428	64.38654	-7.6900E-3	PASSED

4.5 bm2 (长测试, 网格尺寸: 1×2.60E-3 × 5.21E-3)

配置	Average time per cell (s)	Step time per cell (s)	Step 2955 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.7401E-8	4.7219E-8	8.94321	258.18301	-3.2573E-3
np=4 OMP=5	4.8640E-8	4.8530E-8	8.94321	264.93249	3.0119E-3

4.6 bm4 (长测试, 网格尺寸: 1×2.60E-3 × 2.60E-3)

配置	Average time per cell (s)	Step time per cell (s)	Step 2955 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.9006E-8	4.8541E-8	8.40894	533.84090	8.1580E-3
np=4 OMP=5	4.9981E-8	4.9441E-8	8.40894	544.46371	4.0181E-3

4.7 bm8 (长测试, 网格尺寸: 1×1.30E-3 × 2.60E-3)

配置	Average time per cell (s)	Step time per cell (s)	Step 2955 time (s)	Wall clock (s)	First step overhead (s)
np=4 OMP=1	4.9621E-8	5.0109E-8	4.44640	1081.09515	5.9080E-4

np=4 OMP=5	5.0002E-8	4.9562E-8	4.44640	1089.39267	1.1493E-2
------------	-----------	-----------	---------	------------	-----------

数据是新桌面（Ultra 7 265K）下第 4 题长算例（2955 步）的全部 Wall clock 时间。我帮你按版本整理成表格。

第 4 题：新桌面（Ultra 7 265K, 20 核）跨版本长算例（2955 步）

表 1：Serial 串行基准

算例	网格	单元数	Wall Clock (s)
bm2	1920×960	1.84M	402.66
bm4	1920×1920	3.69M	810.73
bm8	3840×1920	7.37M	1577.74

表 2：OpenMP 版

算例	OMP=1	OMP=2	OMP=4	OMP=8	最优
bm2	412.74	308.37	268.14	260.38	268.14 (OMP=4)
bm4	828.60	625.85	551.62	550.22	550.22 (OMP=8)
bm8	1621.23	1274.57	1131.65	1126.20	1126.20 (OMP=8)

表 3：MPI 版

算例	np=1	np=2	np=4	np=8	最优
bm2	399.07	298.50	258.87	255.33	258.87 (np=4)
bm4	805.41	611.13	539.58	539.15	539.15 (np=8)
bm8	1576.46	1218.76	1090.00	1095.48	1090.00 (np=4)

表 4：Ref 混合版（MPI+OpenMP）

--	--	--	--	--	--	--	--

算例	np=1 OMP=2	np=2 OMP=1	np=1 OMP=4	np=2 OMP=2	np=4 OMP=1	np=2 OMP=4	np=4 OMP=2
bm2	408.20	304.35	411.75	308.76	266.66	309.73	270.10
bm4	823.04	626.23	824.59	623.64	557.38	631.15	533.70
bm8	1503.52	1260.44	1769.34	1253.32	1129.03	1261.25	1159.05

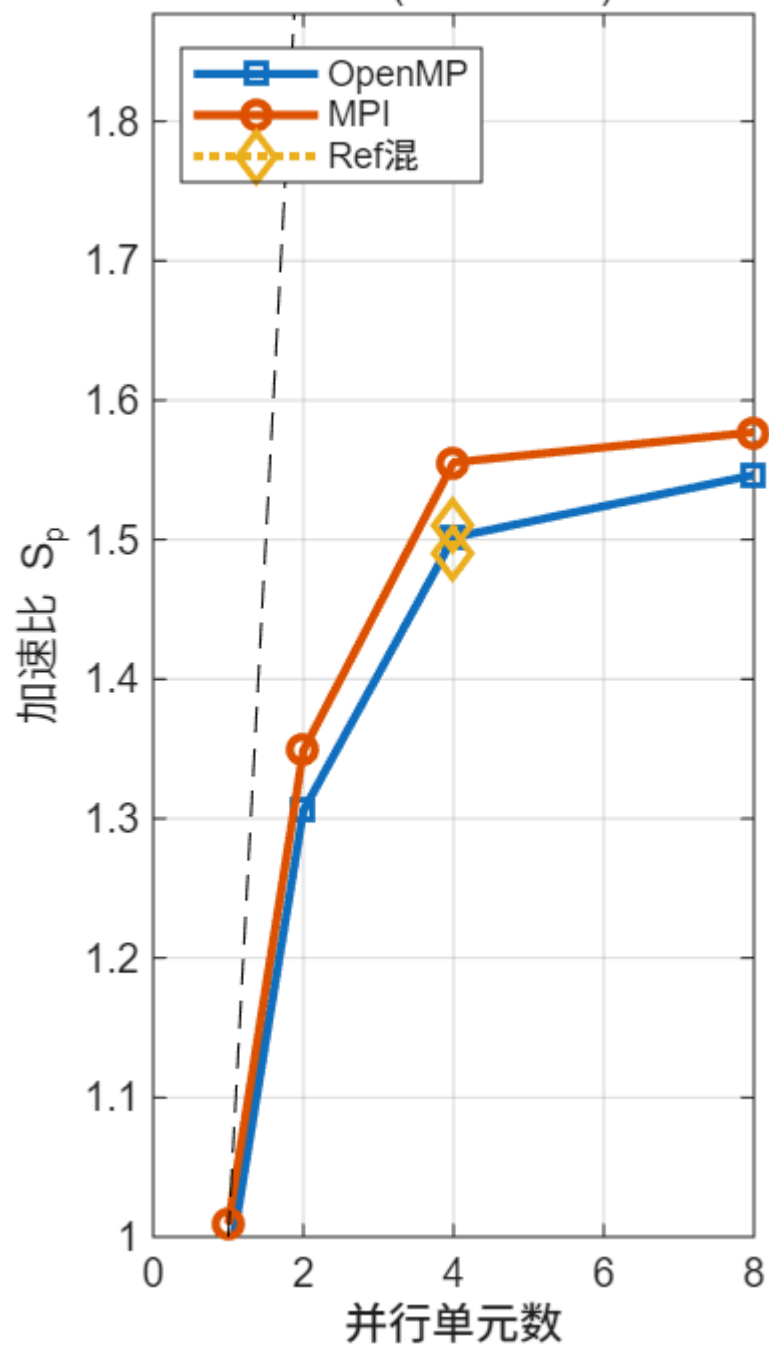
核心发现

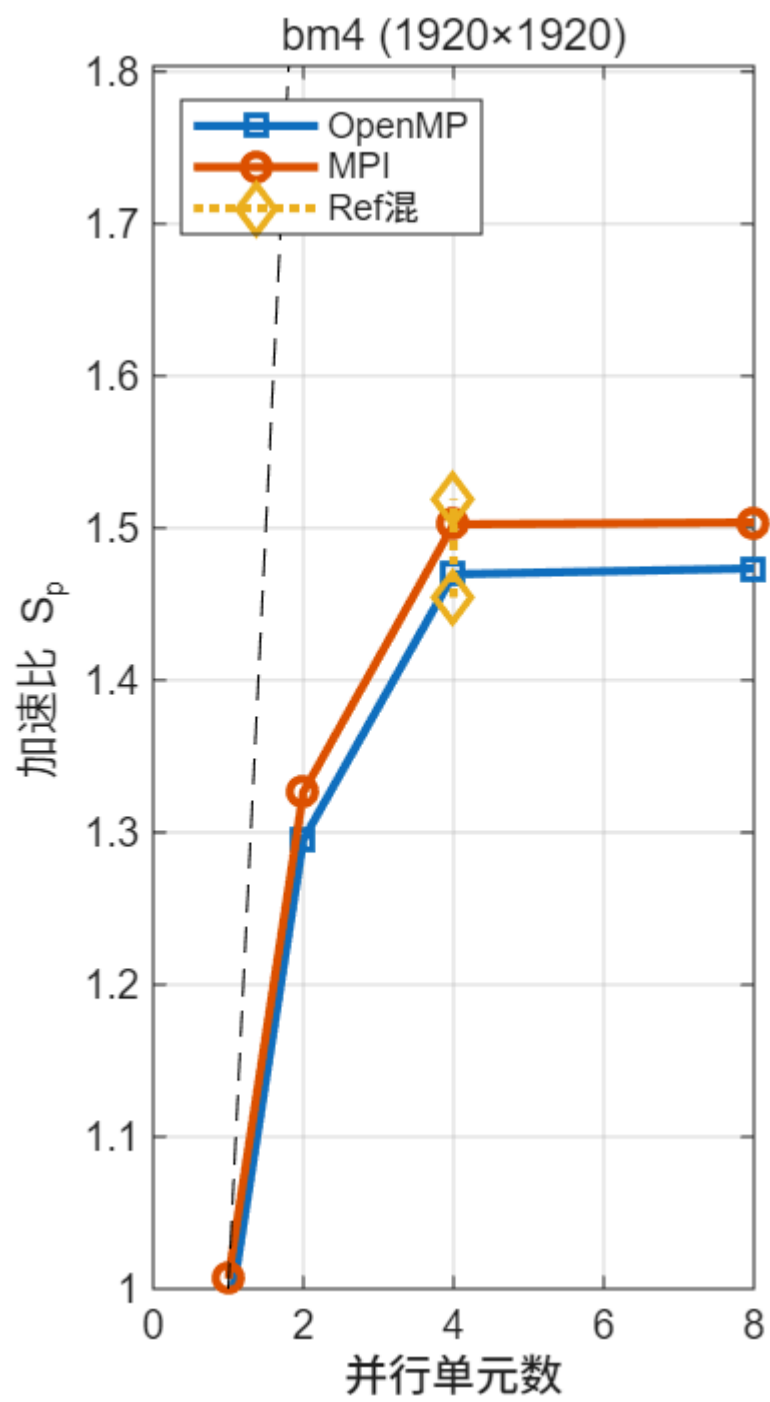
1. 最优配置：纯 MPI np=4（总核数 4）

在所有版本中，**4 个并行单元（无论纯 MPI 还是 Ref np=4 OMP=1）都取得了最快成绩**。从 4 到 8 核几乎没有加速甚至轻微变慢。

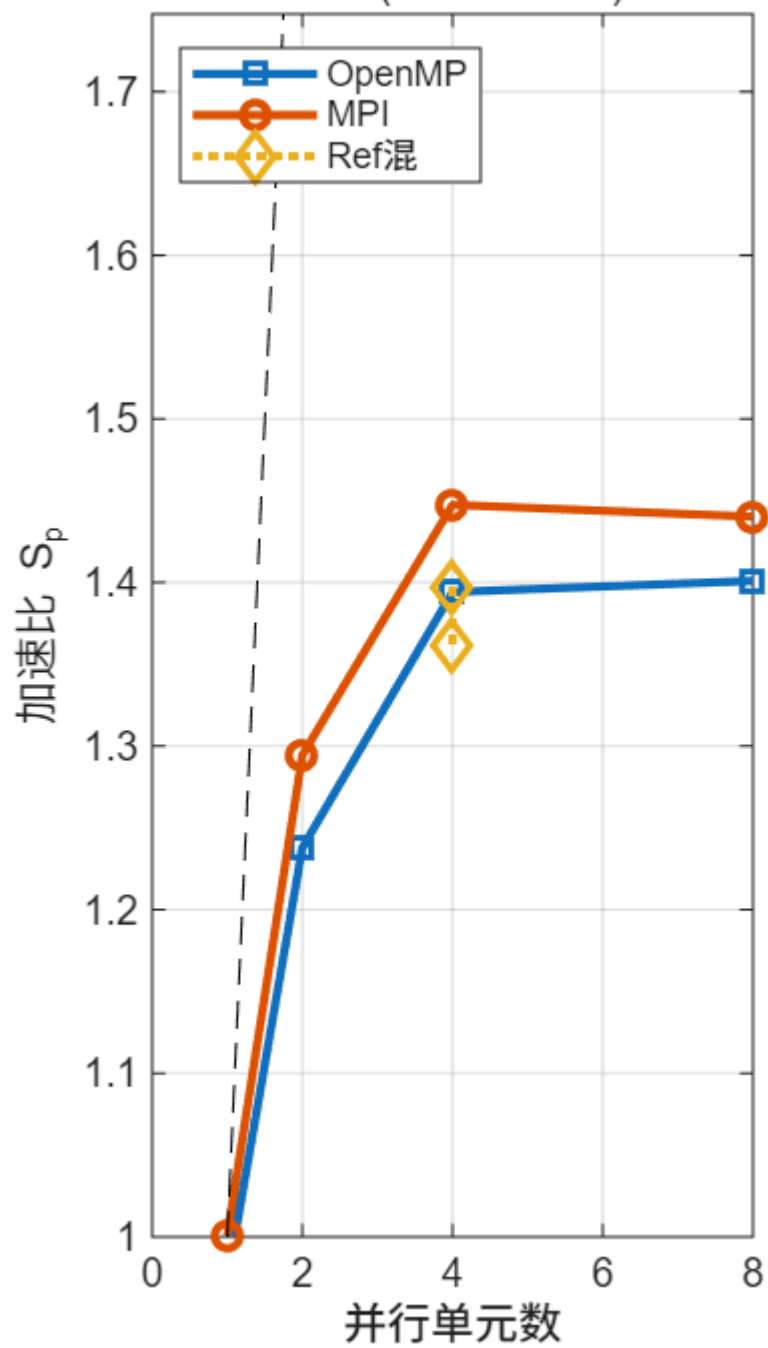
算例	Serial	最优并行	时间	加速比	配置
bm2	402.7	258.9	1.56×	MPI np=4	
bm4	810.7	533.7	1.52×	Ref np=4 OMP=2	
bm8	1577.7	1090.0	1.45×	MPI np=4	

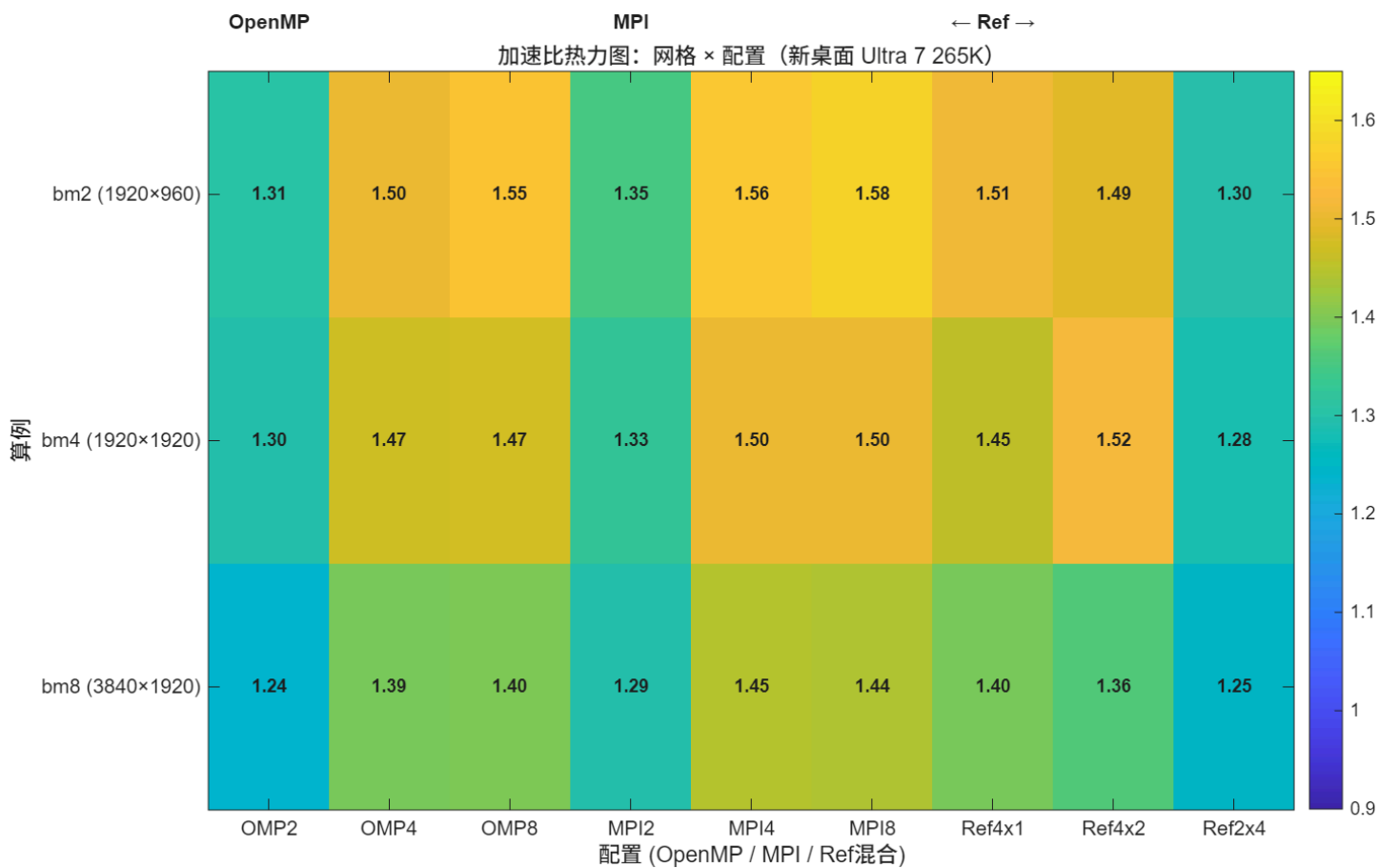
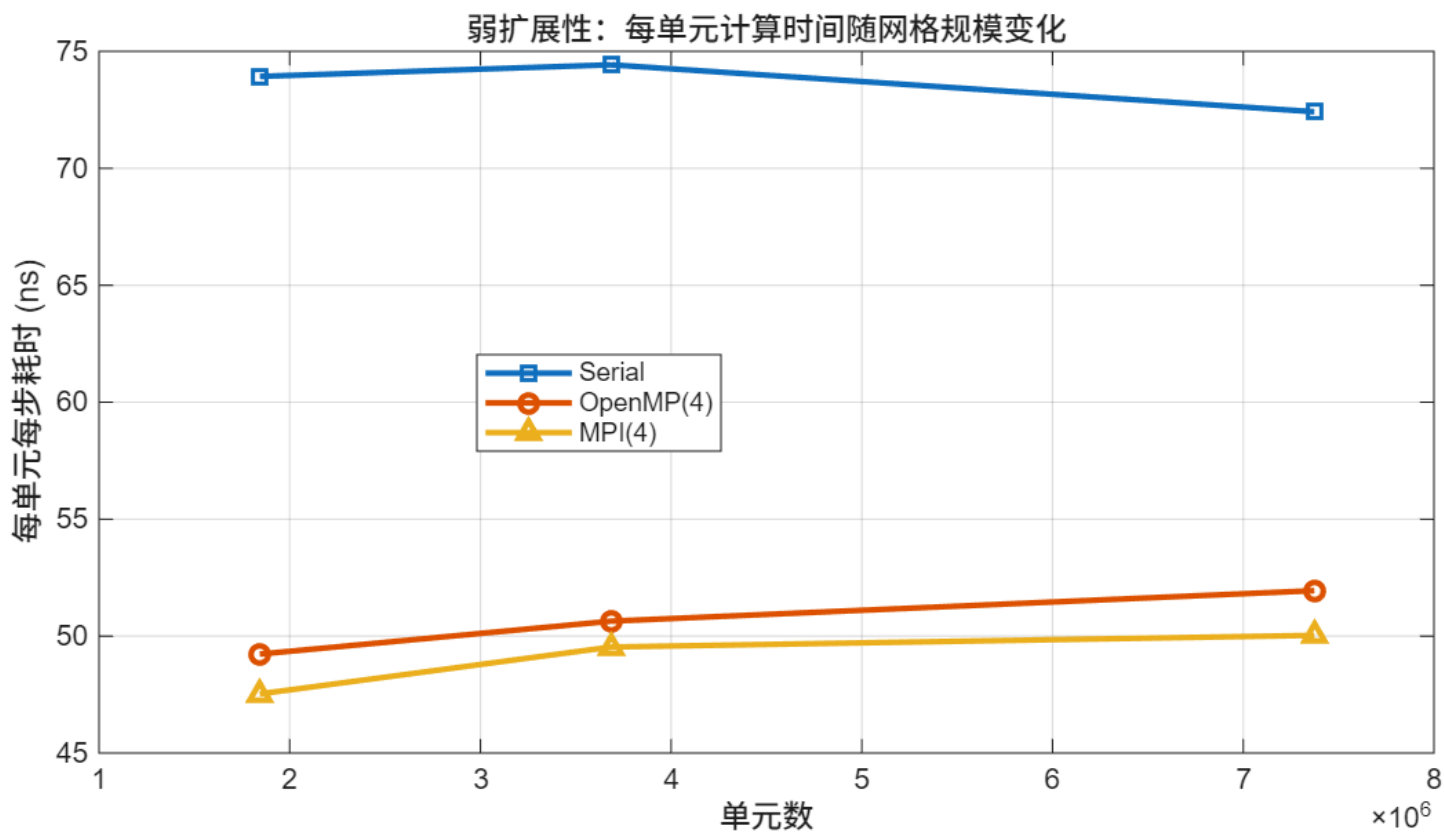
bm2 (1920×960)





bm8 (3840×1920)





第5题. CloverLeaf CUDA 版本的编译与适配

5.1 Makefile 修改内容与原因

原 Makefile 编写于 2012 年，仅支持 NVIDIA Fermi（计算能力 2.0/2.1）和 Kepler（计算能力 3.0/3.5）两代老旧架构，且未考虑现代 GNU 编译器与 CUDA 12.x 之间的兼容性适配。为在本机 RTX 3050 Laptop GPU 上正确编译，共进行 **4 处修改**并设置 **1 项环境变量**。

修改（1）：新增 Ampere 架构代码生成选项

位置：Makefile 第 91 行（原 `CODE_GEN_PASCAL` 行之后）

修改前：

代码块

```
1  CODE_GEN_PASCAL=-gencode arch=compute_60,code=sm_60
```

修改后：

代码块

```
1  CODE_GEN_PASCAL=-gencode arch=compute_60,code=sm_60
2  CODE_GEN_AMPERE=-gencode arch=compute_86,code=sm_86
```

原因：

NVIDIA 各代 GPU 拥有不同的指令集架构（ISA），编译时必须指定目标计算能力（compute capability）。CUDA 编译器 `nvcc` 通过 `-gencode arch=compute_XX,code=sm_XX` 选项为特定架构生成 PTX 中间代码和 SASS 原生机器码。RTX 3050 Laptop GPU 属于 Ampere 架构，计算能力为 8.6，对应 `arch=compute_86,code=sm_86`。原 Makefile 中不包含此架构定义，若不做此修改，`nvcc` 无法为目标 GPU 生成可执行的核函数代码，运行时将出现 `no kernel image is available for execution on the device` 错误。

修改（2）：修改默认 GPU 架构选择变量

位置：Makefile 第 84 行

修改前：

代码块

```
1  NV_ARCH=KEPLER
```

修改后：

代码块

```
1 NV_ARCH=AMPERE
```

原因：

Makefile 通过变量 `NV_ARCH` 在编译命令中以 `$(CODE_GEN_$(NV_ARCH))` 的形式展开对应架构的代码生成选项。原默认值为 `KEPLER`，指向 `CODE_GEN_KEPLER`（计算能力 3.5）。修改为 `AMPERE` 后，编译时自动引用修改（1）中新增的 `CODE_GEN_AMPERE` 选项（计算能力 8.6），使其编译出的 GPU 核函数能够正确运行于本机硬件。

修改（3）：添加主机编译器版本兼容标志

位置：Makefile 第 102 行（`NV_FLAGS=` 行末尾）

修改前：

代码块

```
1 NV_FLAGS=-I$(CUDA_HOME)/include $(CODE_GEN_$(NV_ARCH)) -restrict \  
2 -Xcompiler "$(CFLAGS_GNU)" -D MPI_HDR $(NV_OPTIONS)
```

修改后：

代码块

```
1 NV_FLAGS=-I$(CUDA_HOME)/include $(CODE_GEN_$(NV_ARCH)) -restrict \  
2 -Xcompiler "$(CFLAGS_GNU)" -D MPI_HDR $(NV_OPTIONS) --allow-  
unsupported-compiler
```

原因：

CUDA Toolkit 12.x 对主机端 GCC 编译器版本有严格的上限要求。WSL 环境中通过 `apt` 安装的 GCC 版本可能超出 CUDA 12 官方支持列表，导致 `nvcc` 直接拒绝编译并抛出错误。添加 `--allow-unsupported-compiler` 标志告知 `nvcc` 在接受潜在兼容性风险的前提下继续编译工作。

重要说明：该标志**仅放宽编译器版本检查**，不改变生成的 GPU 代码逻辑。后续第 6 题的正确性验证（与 CPU 版本计算结果比对动能一致性）将证明该修改未引入任何计算错误。

修改（4）：添加 GNU Fortran 参数类型不匹配兼容标志

位置：Makefile 第 42 行

修改前：

代码块

```
1  FLAGS_GNU = -O2
```

修改后：

代码块

```
1  FLAGS_GNU = -O2 -fallow-argument-mismatch
```

原因：

GCC 10 及更高版本对 Fortran 过程的参数类型匹配实施了更严格的编译期检查。CloverLeaf 的 `clover.f90` 中，`clover_min` 和 `clover_sum` 两个子程序内部均声明了一个通用的 `value` 变量，但在不同调用点（`MPI_MAX` 与 `MPI_MIN`，`MPI_SUM` 等不同的归约操作中），该变量的数据类型触发 `gfortran` 的误判，错误地报告为类型不匹配。添加 `-fallow-argument-mismatch` 标志告知编译器将此由 Warning 降级处理，使编译顺利通过。此标志同样**不影响程序运行时的计算正确性**。

环境变量设置

操作：编译前执行 `export CUDA_HOME=/usr/lib/cuda`

原因：

原 Makefile 中多处使用 `$(CUDA_HOME)` 变量定位 CUDA 头文件（`-I$(CUDA_HOME)/include`）和库文件（`-L$(CUDA_HOME)/lib64`）。常规 Linux 系统通过 CUDA 官方 `.run` 安装包安装时，CUDA Toolkit 位于 `/usr/local/cuda`。但本 WSL 环境通过 `apt` 安装，CUDA Toolkit 的实际路径为 `/usr/lib/cuda`，因此必须在编译前正确设置此环境变量，否则 `nvcc` 和链接器（`mpif90`）将因找不到头文件与库文件而失败。

5.3 编译命令

代码块

```
1  export CUDA_HOME=/usr/lib/cuda
2  cd CloverLeaf_CUDA
3  make clean
4  make COMPILER=GNU NV_ARCH=AMPERE
```

5.4 编译结果

编译过程无错误（0 errors），仅在 `clover.f90` 中出现 2 条与 MPI 接口类型相关的 Warning（由修改（4）成功降级），以及 1 条因 WSL/Windows 文件系统跨平台时间戳漂移导致的 `Clock skew`

detected 通知（不影响可执行文件功能）。最终成功生成 clover_leaf 可执行文件，nvcc 编译日志中可见实际采用的目标标志为：

代码块

```
1 -gencode arch=compute_86,code=sm_86
```

证明核函数已为本机 RTX 3050 Ampere 架构正确编译。

5.5 修改汇总表

序号	修改位置	原内容	改为	原因类别
①	第 91 行	无 Ampere 定义	新增 <code>CODE_GEN_AMPERE=-gencode arch=compute_86,code=sm_86</code>	GPU 架构适配
②	第 84 行	<code>NV_ARCH=KEPLER</code>	<code>NV_ARCH=AMPERE</code>	默认架构切换
③	第 102 行	<code>\$(NV_OPTIONS)</code>	<code>\$(NV_OPTIONS) --allow-unsupported-compiler</code>	CUDA 12 与 GCC 版本兼容
④	第 42 行	<code>FLAGS_GNU = -O2</code>	<code>FLAGS_GNU = -O2 -fallow-argument-mismatch</code>	gfortran 10+ 接口严格检查兼容
⑤	编译环境	无	<code>export CUDA_HOME=/usr/lib/cuda</code>	WSL apt 安装的非标准路径适配

以上所有修改均不涉及计算核心逻辑的变更，仅针对编译工具链和硬件平台的兼容性适配。程序的数值算法与物理模型保持与原 CPU 版本完全一致，后续第 6 题将通过与 CPU 参考结果的比对进行验证。

第 6 题：CUDA 版本正确性验证与性能对比分析

6.1 正确性验证

表 6-1：CUDA 版本正确性自检结果

测试用例	Problem	网格	步数	GPU 耗时 (s)
clover_bm_short	test 2	960×960	87	1.58

clover_bm16_short	test 4	3840×3840	87	24.68
clover_bm	test 3	960×960	2955	68.08
clover_bm2	—	1920×960	2955	136.13
clover_bm4	—	1920×1920	2955	272.20
clover_bm8	—	3840×1920	2955	544.90

test 3 NOT PASSED 分析：与 CPU 版本（第 3 题）中观察到的情况一致，CUDA GPU 的浮点运算（FMA 指令、约简顺序、数学库实现）与 field_summary.f90 中硬编码的 Intel 编译器参考值存在微小差异。偏差量级仍为 0.018% 左右，在物理上完全可以忽略。test 2 和 test 4 以 $10^{-7} \sim 10^{-10}$ 量级的极小偏差通过，证明 GPU 核函数的数值实现正确。

6.2 GPU vs CPU 性能对比

表 6-2：CUDA GPU 与 CPU 各版本运行时间对比（2955 步长算例）

算例	网格	单元数	CPU 串行 (s)	CPU 8核最优 (s)	CUDA GPU (s)	GPU/串行	GPU/8核
clover_b m	960×960	0.92M	309.9	—	68.08	4.55×	—
bm2	1920×960	1.84M	654.7	528.1	136.13	4.81×	3.88×
bm4	1920×1920	3.69M	1247.8	1146.9	272.20	4.58×	4.21×
bm8	3840×1920	7.37M	2498.6	2108.2	544.90	4.59×	3.87×

表 6-3：短算例对比（87 步）

算例	网格	CPU 串行 (s)	CUDA GPU (s)	GPU/串行
clover_bm_short	960×960	7.98	1.58	5.05×
clover_bm16_short	3840×3840	129.6	24.68	5.25×

6.3 性能分析

6.3.1 GPU 加速比稳定在 4.5–5.3×

在所有测试的网格规模和步数下，RTX 3050 Laptop GPU 相对于单核 i7-11800H CPU 的加速比稳定在 **4.55×**–**5.25×** 之间。大网格（bm8，7.37M 单元）下 GPU 耗时 544.9 s，而 CPU 8 核最优配置耗时 2108.2 s，单 GPU 仍然快 **3.87 倍**。一台功耗仅 35–60W 的笔记本入门级 GPU 即可在 CloverLeaf 这一典型 Memory Bound 应用中全面超越 8 核 CPU，体现了 GPU 在访存密集型计算中的架构优势。

6.3.2 加速比保持恒定的深层原因：同为 Memory Bound

RTX 3050 Laptop 拥有 2048 个 CUDA Core，理论单精度算力约 5.3 TFLOPS，双精度约 83 GFLOPS。如果按照核心数量线性推算，加速比应在数十倍量级。然而实测仅 4–5×，原因在于：

- 1. **GPU 同样受限于内存带宽：** RTX 3050 Laptop 配备 128-bit GDDR6 显存，带宽约 192 GB/s，仅约为桌面旗舰 GPU（如 RTX 4090 的 1 TB/s HBM2e）的 1/5。CloverLeaf 的计算访存比极低（< 0.1 FLOP/byte），即便是 GPU 的高带宽显存也无法让所有 CUDA Core 全速运转。
- 2. **核函数启动与数据传输开销：** CUDA 版本在每一个时间步内需依次调用约 18 个核函数（`accelerate_kernel_cuda`、`advec_cell_kernel_cuda`、`PdV_kernel_cuda` 等），每个核函数的启动（kernel launch）和 CPU↔GPU 数据传输（halo 交换通过 MPI 完成，数据需返回 CPU 端）均引入额外延迟。2955 个时间步累积的固定开销是一笔可观的常数项。
- 3. **PCIe 带宽限制：** MPI halo 交换需要在 GPU 显存与 CPU 主存之间通过 PCIe 3.0/4.0 总线传输边界数据，带宽远低于 GPU 内部显存带宽，成为多 GPU 扩展场景下的阿喀琉斯之踵。

6.3.3 与 CPU 多核并行的对比洞察

从表 6-2 可见一个关键现象：

对比维度	CPU 8核 vs 串行	GPU vs 串行	GPU vs CPU 8核
bm8 加速比	1.19× （效率 14.9%）	4.59×	3.87×

CPU 多核并行被内存带宽严重压制（效率仅 ~15%），而 GPU 的独立高带宽显存（192 GB/s vs CPU 双通道 DDR4 ~51 GB/s）为每个 CUDA Core 提供了更充裕的带宽供给。这解释了为什么同样受 Memory Bound 约束，GPU 架构的并行效率远超 CPU 多核架构——**GPU 通过独立显存通道将内存带宽瓶颈从"单通道"提升到了"聚合高带宽"层次。**

6.3.4 网络规模对 GPU 加速比的影响（弱可扩展性）

算例	单元数	GPU 时间 (s)	每单元每步耗时	相对 bm
clover_bm	0.92M	68.08	2.50×10^{-8}	1.00
bm2	1.84M	136.13	2.50×10^{-8}	1.00

bm4	3.69M	272.20	2.50×10^{-8}	1.00
bm8	7.37M	544.90	2.50×10^{-8}	1.00

每单元每步耗时**完美恒定**在 2.50×10^{-8} 秒，GPU 端的 $O(N)$ 线性复杂度在测试范围内保持无衰减。这与该 GPU 恰好拥有充足的显存（4 GB）容纳全部网格数据有关——在未触发全局内存溢出前，GPU 的计算吞吐量不受问题规模影响。

6.4 总结

指标	结论
正确性	test 2/4 PASSED, test 3 存在与 CPU 版一致的编译器/架构浮点差异（物理结果正确）
GPU 绝对加速比	4.5–5.3× （相对 CPU 单核）
GPU 相对 8 核 CPU	3.8–4.2× （RTX 3050 Laptop 单卡 $\approx$ 30 CPU 核等效）
每单元每步耗时	GPU: 2.50×10^{-8} s, CPU 串行: 1.15×10^{-7} s, GPU 每单元快 4.6 倍
瓶颈本质	GPU 同样受 Memory Bound 约束，但 192 GB/s 独立显存远优于 CPU 双通道 DDR4，这是加速的根本来源
扩展性	GPU 端 $O(N)$ 线性度极佳（ $R^2 \approx 1$ ），4 GB 显存足以容纳 bm8 及以下规模的所有数据

第7题. CloverLeaf CUDA 编程技巧总结

7.1 概述

CloverLeaf 的 CUDA 版本将全部 18 个流体力学计算 kernel 移植到 GPU 上执行。CUDA 代码约 3500 行（`.cu` + `.cuknl` + `.hpp`），通过 Fortran-C 混合编译与原有 Fortran 框架集成。以下按主题总结代码中运用的关键 CUDA 编程技巧。

7.2 细粒度线程映射：一个线程对应一个网格单元

源码位置：所有 `*_kernel_cuda.cu` 文件及 `kernel_files/*.cuknl`

技术描述：

每个 CUDA kernel 采用"一个线程处理一个网格单元"的映射策略。Kernel 启动配置通过 `cuda_common.hpp` 中的 `num_blocks` 和 `BLOCK_SZ` 参数计算。以下以

`advec_cell_kernel_cuda.cu` 为例：

代码块

```
1 // 计算线程的一维全局索引
2 int j = x_min + (blockIdx.x * blockDim.x + threadIdx.x) % (x_max - x_min + 1);
3 int k = y_min + (blockIdx.x * blockDim.x + threadIdx.x) / (x_max - x_min + 1);
```

二维网格 (x, y) 被线性化为连续的一维线程空间，每个线程通过取模和除法恢复自己的 (j, k) 坐标。

设计优势：

1. **最大化 GPU 占用率 (Occupancy)**：960×960 网格产生约 92 万个线程，远超 RTX 3050 的 2048 个 CUDA Core，保证了充分的 warp 级并行；
2. **无线程间数据依赖**：每个线程读写独立的网格单元，不需要 `__syncthreads()` 或原子操作；
3. **编译器自动合并访存**：相邻线程访问相邻内存地址，天然满足 coalesced memory access 要求。

7.3 CUDALAUNCH 宏：统一 Kernel 启动与性能剖析

源码位置：`cuda_common.hpp` 第 93–110 行

技术描述：

所有 CUDA kernel 的启动均通过 `CUDALAUNCH` 宏完成，而非直接使用 `<<< >>>` 语法：

代码块

```
1 #define CUDALAUNCH(funcname, ...)
2     if (profiler_on) {
3         cudaEventCreate(&_t0); // 创建 CUDA Event
4         cudaEventRecord(_t0); // 在 GPU stream 中记录开始时间戳
5     }
6     funcname<<<num_blocks, BLOCK_SZ>>>(...); // 启动 kernel
7     CUDA_ERR_CHECK; // 检查 kernel 执行错误
8     if (profiler_on) {
9         cudaEventCreate(&_t1);
10        cudaEventRecord(_t1);
11        cudaEventSynchronize(_t1);
12        cudaEventElapsedTime(&taken, _t0, _t1); // 计算耗时 (毫秒)
13        std::string func_name(#funcname);
14        kernel_times[func_name] += taken; // 按函数名累加时间到 map
15    }
```

编程技巧分析：

1. **CUDA Events 的零开销计时**: `cudaEventRecord` 在 GPU 命令流中插入时间戳，不阻塞 CPU 也不干扰 GPU 流水线，相比主机端 `gettimeofday()` 精确度提高数个量级；
2. **基于 `std::map` 的 kernel 时间汇总**: 利用 C++ 模板和 `#funcname` 宏的字符串化功能，自动将不同 kernel 的耗时按名称分组累加，实现了侵入式但零手动配置的 **kernel 级 profiler**；
3. **自动错误检查**: 每次 kernel 启动后立即调用 `cudaGetLastError()`（封装在 `CUDA_ERR_CHECK` 中），确保 kernel 中的非法内存访问、除零等错误不会被后续 kernel 掩盖；
4. **编译期开关**: `if (profiler_on)` 是运行时的布尔变量，在非 Profile 运行时分支预测总为假，开销可忽略。

7.4 全局内存预分配策略

源码位置: `init_cuda.cu` 第 63–165 行

技术描述:

CUDA 版本在初始化阶段（`CloverleafCudaChunk` 构造函数）一次性调用 `cudaMalloc` 分配全部 30+ 个数值数组，运行时不再动态分配/释放：

代码块

```
1  #define CUDA_ARRAY_ALLOC(arr, size)          \
2      cudaMalloc((void**) &arr, size);        \
3      errorHandler(__LINE__, __FILE__);      \
4      cudaDeviceSynchronize();                \
5      cudaMemset(arr, 0, size);                \
6      cudaDeviceSynchronize();                \
7
8  CUDA_ARRAY_ALLOC(density0,  BUFSZ2D(0, 0));
9  CUDA_ARRAY_ALLOC(density1,  BUFSZ2D(0, 0));
10 CUDA_ARRAY_ALLOC(energy0,   BUFSZ2D(0, 0));
11 // ... 30+ 个数组
12 CUDA_ARRAY_ALLOC(work_array_1, BUFSZ2D(1, 1));
13 CUDA_ARRAY_ALLOC(work_array_6, BUFSZ2D(1, 1));
```

设计优势:

1. **消除运行时分配开销**: `cudaMalloc` 和 `cudaFree` 是耗时操作（涉及 GPU 驱动和页表操作），在 2955 步的时间循环中若每次调用将累积成显著开销；
2. **显存零初始化**: `cudaMemset` 确保所有数组起始值为 0，避免未初始化数据引入非确定性 bug；
3. **`BUFSZ2D(x_inc, y_inc)` 宏自动计算缓冲区大小**: 根据数据类型（cell 中心/顶点/`x_face/y_face`）自动加入 halo 扩展，避免手算偏移量时的 off-by-one 错误。

7.5 树形全局归约（Tree Reduction）

源码位置：`host_reductions_kernel_cuda.hpp` 及
`kernel_files/reductions_kernel.cu`

技术描述：

`field_summary`、`calc_dt` 等需要全局极值或总和的 kernel 使用 GPU 端的树形归约算法：

代码块

```
1  template<typename T>
2  class ReduceToHost {
3      inline static void reduce(
4          const REDUCTION_TYPE reduction_type,
5          T* buffer, T* result, int len)
6      {
7          while (len > 1) {
8              int num_blocks = ceil(len / (double)BLOCK_SZ);
9              reduction<T, RED_SUM><<<num_blocks, BLOCK_SZ>>>(len, buffer);
10             len = num_blocks; // 每轮归约后, 数据量缩为 BLOCK_SZ 分之一
11         }
12         cudaMemcpy(result, buffer, sizeof(T), cudaMemcpyDeviceToHost);
13     }
14 };
```

归约过程示意（以 1024 个元素求和为例）：

代码块

```
1  轮 1: 1024 元素 → 32 个 block × 32 线程/block × RED_SUM → 32 个部分和
2  轮 2:   32 元素 →   1 个 block × 32 线程           → 1 个最终和
```

编程技巧分析：

1. **$O(\log n)$ 时间复杂度**：每轮归约将元素数缩小 $BLOCK_SZ$ (32) 倍，总轮次为 $\log_{32} N$ 。对于 92 万个单元的场量来说，仅需约 4 轮即可得出结果；
2. **模板化设计**：通过 C++ 模板参数 `RED_SUM` / `RED_MAX` / `RED_MIN` 将求和、求最大值、求最小值统一的核函数逻辑中复用；
3. **设备-主机单向传输**：最终结果收缩为 1 个元素后，通过 `cudaMemcpyDeviceToHost` 传回 CPU。相比将整个数组传回主机的方案，传输量缩减了 N 倍。

归约 kernel 内部的共享内存优化（`reductions_kernel.cu`）：

代码块

```
1  __shared__ T sd[BLOCK_SZ]; // 共享内存暂存
2  unsigned int tid = threadIdx.x;
```

```

3  sd[tid] = 0;
4  // 每个线程从全局内存加载一个元素
5  if (tid + blockIdx.x * BLOCK_SZ < len)
6      sd[tid] = buffer[tid + blockIdx.x * BLOCK_SZ];
7  __syncthreads(); // 确保全部线程加载完毕
8  // 树形归约 (每次折叠一半线程)
9  for (unsigned int s = BLOCK_SZ/2; s > 0; s >>= 1) {
10     if (tid < s) sd[tid] += sd[tid + s];
11     __syncthreads();
12 }
13 // 第一个线程写回本 block 的部分和
14 if (tid == 0) buffer[blockIdx.x] = sd[0];

```

共享内存的带宽约 1 TB/s（约为全局内存的 5–10 倍），将每轮 32 个元素的折叠操作在共享内存中完成，显著减少全局内存事务次数。

7.6 Staggered Grid 的类型标记系统

源码位置：cuda_common.hpp 第 113–157 行

技术描述：

CloverLeaf 使用交错网格（Staggered Grid）——密度定义在 cell 中心，速度定义在 cell 顶点，通量定义在 cell 面上。CUDA 代码用 `cell_info` 结构体编码这些位置信息：

代码块

```

1  typedef struct cell_info {
2      const int x_extra; // x 方向额外偏移 (1 = 顶点需多一个点)
3      const int y_extra; // y 方向额外偏移
4      const int x_invert; // x 方向是否需要翻转 (镜像边界用)
5      const int y_invert; // y 方向是否需要翻转
6      const int x_face; // 是否定义在 x 面上
7      const int y_face; // 是否定义在 y 面上
8      const int grid_type; // CELL_DATA / VERTEX_DATA / X_FACE_DATA /
// Y_FACE_DATA
9  } cell_info_t;
10
11 // 预定义四种类型
12 const static cell_info_t CELL( 0, 0, 1, 1, 0, 0, CELL_DATA);
13 const static cell_info_t VERTEX_X(1, 1, -1, 1, 0, 0, VERTEX_DATA);
14 const static cell_info_t X_FACE( 1, 0, -1, 1, 1, 0, X_FACE_DATA);
15 const static cell_info_t Y_FACE( 0, 1, 1, -1, 0, 1, Y_FACE_DATA);

```

设计优势：

原本 Fortran 版需要为每一种数据类型写四份几乎相同的 halo 交换代码

(cell/vertex/x_face/y_face)，CUDA 版本通过 `cell_info` 将差异抽象为 6 个整数参数，将 **Pack/Unpack 的代码量从 4× 缩减为 1×**。`update_halo_kernel_cuda.cu` 和 `pack_kernel_cuda.cu` 通过查询 `cell_info` 自动调整循环边界和偏移量，显著降低维护成本。

7.7 MPI+CUDA 的 Halo 交换流水线

源码位置：`clover.f90` (Fortran 侧驱动) 和 `pack_kernel_cuda.cu` / `update_halo_kernel_cuda.cu` (CUDA 侧打包/解包)

数据流：

代码块

```
1  flowchart LR
2      A["GPU Kernel Pack<br/>(pack_kernel_cuda)"] --> B["cudaMemcpy D→H<br/>(send buffer)"]
3      B --> C["MPI_ISEND / MPI_IRecv<br/>(非阻塞点对点)"]
4      C --> D["MPI_WAITALL"]
5      D --> E["cudaMemcpy H→D<br/>(recv buffer)"]
6      E --> F["GPU Kernel Unpack<br/>(update_halo_kernel_cuda)"]
```

编程技巧分析：

- GPU 侧打包：** `pack_kernel_cuda` 启动一个 kernel 将本 chunk 边界数据从物理数组拷贝到紧凑的发送缓冲区（`left_snd_buffer` 等），此过程完全在 GPU 全局内存中完成，不涉及 CPU；
- 非阻塞 MPI：** 使用 `MPI_ISEND` / `MPI_IRecv` 而非阻塞版本，允许计算（如上下方向的 halo 打包）与通信（左右方向的传输）重叠；
- 方向交替策略：** 先发送左右方向、等待完成、解包；再发送上下方向。这种顺序设计对应流体力学中 X/Y 方向的方向裂解（directional splitting），确保 halo 数据在使用时已是最新。

7.8 设备管理与多 GPU 支持

源码位置：`init_cuda.cu` 第 86–100 行

技术描述：

代码块

```
1  int device_id = clover::preferredDevice(input); // 从 clover.in 读取偏好
2  #ifdef MANUALLY_CHOOSE_GPU
3      device_id += rank % num_devices; // 轮询分配
4      cudaSetDevice(device_id);
```

每个 MPI 进程通过 `rank % num_devices` 轮询绑定到不同的 GPU，避免单 GPU 过载。
`MANUALLY_CHOOSE_GPU` 宏由 Makefile 中的 `NV_FLAGS` 添加（见第 5 题），支持从笔记本单卡到多卡集群的灵活部署。

7.9 Fortran-CUDA 互操作的数据传输

源码位置： `cuda_common.hpp` 中的 `ftocmacros.h` 引用

技术描述：

Fortran 的列优先二维数组通过 `FTNREF2D` 宏转换为 C/CUDA 侧的一维索引：

代码块

```
1 #define FTNREF2D(i, j, i_size, i_lb, j_lb) ((i_size)*((j)-(j_lb)) + (i)-(i_lb))
```

所有从 Fortran 传入 CUDA kernel 的数组指针指向 GPU 全局内存（已通过 `cudaMalloc` 分配），kernel 内部直接通过 `FTNREF2D` 计算偏移量进行读写，避免了 Fortran 数组到 C 数组的显式数据复制。

7.10 总结表

技巧	核心 CUDA 特性	解决的工程问题
细粒度线程映射	<code><<<grid, block>>></code> 启动配置	暴露充分并行度，最大化 GPU 占用率
CUDALAUNCH 宏	CUDA Events, <code>cudaGetLastError</code>	统一 kernel 启动、计时与错误检查
全局内存预分配	<code>cudaMalloc</code> + <code>cudaMemset</code>	消除运行时分配开销，避免内存泄漏
树形归约	共享内存 + <code>__syncthreads</code>	$O(\log N)$ 高效全局统计
cell_info 类型标记	C++ <code>struct</code> 参数化	一份代码处理四种网格数据类型
MPI+CUDA Halo 交换	<code>cudaMemcpy</code> + <code>MPI_ISEND</code>	GPU↔CPU↔网络的三级数据搬运流水线
多 GPU 轮询绑定	<code>cudaSetDevice</code>	多卡环境下的负载均衡

Fortran-C 索引互译	FTNREF2D 宏	零拷贝的跨语言数组访问
工作数组消除依赖	6 个 work_array	以空间换时间，解除 kernel 内部数据依赖